MFA OTP - Backend Development Document

API Endpoints

All endpoints are **pre-login** (no JWT required) under `/mfa_otp/`.

#	Endpoint	Purpose
1	`POST /mfa_otp/init_session`	Create a new MFA session
2	`POST /mfa_otp/send_otp`	Send OTP to registered mobile
3	`POST /mfa_otp/verify_otp`	Verify OTP entered by user

Flow

```
Client                          Server                        Integration Broker
  |                               |                                   |
  |--- init_session (emplid) ---->|                                   |
  |<-- mfaSessionId, status: S ---|                                   |
  |                               |  [Session: INITIALIZED]           |
  |                               |                                   |
  |--- send_otp (emplid, ------->|--- IB: SRV_NM_MFA_SEND_OTP ----->|
  |    mfaSessionId)              |<-- status: S/F ------------------|
  |<-- status, mfaSessionId -----|                                  |
  |                               |  [Session: OTP_SENT]              |
  |                               |                                   |
  |--- verify_otp (emplid, ----->|--- IB: SRV_NM_MFA_VERIFY_OTP --->|
  |    mfaSessionId, otp)         |<-- status: S/F ------------------|
  |<-- status, timeOutVal -------|                                  |
  |                               |  [Session: VERIFIED -> consumed] |
```

Session State Machine

```
INITIALIZED  -->  OTP_SENT  -->  VERIFIED  -->  consumed (deleted)
     |              |              |
     |              |              +-- consumeVerifiedSession() removes from store
     |              +-- stays OTP_SENT on failed verify or resend
     +-- first send_otp transitions here
```

- **Storage:** In-memory `ConcurrentHashMap` (per server instance)

- **Storage**: In-memory `ConcurrentHashMap` (per server instance)
- **Thread safety**: `ConcurrentHashMap.compute()` + `synchronized` on session object
- **State transitions**: Atomic, only on IB success (`status == "S"`)

Limits & Timeouts

Session

Property	Value	Description
`MFA_SESSION_TTL_MINUTES`	5	Session expires 5 minutes after creation
`MFA_SESSION_CLEANUP_INTERVAL_SECONDS`	60	Daemon thread cleans expired sessions every 60s

OTP Send (Resend)

Property	Value	Description
`MFA_MAX_OTP_SENDS`	3	Max OTP sends per session (1 initial + 2 resends)
`MFA_OTP_WINDOW_MINUTES`	10	Sliding window for the send count

- OTP send count is **reset on each** `init_session` call, so each new flow gets a fresh 3 sends.
- No cooldown/delay between resends.
- `send_otp` accepts both `INITIALIZED` and `OTP_SENT` states (supports resend without reinitializing).

OTP Verify

Property	Value	Description
`MFA_MAX_VERIFY_ATTEMPTS`	3	Max wrong OTP attempts per session

- OTP correctness is checked **first** (IB call happens before incrementing counter).
- Correct OTP on any attempt (1st, 2nd, or 3rd) returns success.
- Counter increments only on **wrong** OTP.
- On 3rd wrong attempt: session is invalidated, user must start over from `init_session`.
- A **pre-check** rejects requests immediately if attempts are already exhausted (without calling IB).
- Failure responses include `remainingAttempts` count.

Session Init Rate Limit

Property	Value	Description
`MFA_MAX_SESSION_INITS`	5	Max `init_session` calls per emplid per window
`MFA_SESSION_INIT_WINDOW_MINUTES`	10	Sliding window for init count

Cleanup

Property	Value	Description
`MFA_RATE_LIMIT_CLEANUP_INTERVAL_SECONDS`	60	Daemon cleans expired rate limit entries

Request / Response Format

All requests and responses support **AES+RSA encryption**. If the request contains `data` and `key` fields, it is treated as encrypted; otherwise plain JSON.

Request Payload (all 3 endpoints)

```
{
  "emplid": "123456",
  "mfaSessionId": "uuid-string",
  "otp": "1234"
}
```

> `mfaSessionId` is required for `send_otp` and `verify_otp`. `otp` is required only for `verify_otp`.

Success Response

```
{
  "auth": { "auth": "", "csrf": "" },
  "status": "S",
  "message": "...",
  "mfaSessionId": "uuid-string",
  "timestamp": 1234567890,
  "timeOutVal": "5000"
}
```

> `timeOutVal` is only present in `verify_otp` success response. Value is fetched from DB (`MFA_REG_TIMEOUT_VAL`), defaults to `"5000"`.

Failure Response (wrong OTP)

```
{
  "auth": { "auth": "", "csrf": "" },
  "status": "F",
  "message": "...",
  "mfaSessionId": "uuid-string",
  "remainingAttempts": 2,
  "timestamp": 1234567890
}
```

Error Response (validation/server error)

```
{
  "auth": { "auth": "", "csrf": "" },
  "error": { "code": "500", "msg": "...", "desc": "...", "reso": "" }
}
```

Key Files

File	Purpose
`controller/MfaOtpController.java`	REST controller - 3 endpoints, orchestrates entire flow
`component/MfaSessionManager.java`	Session state machine, TTL, verify attempt tracking
`component/MfaRateLimiter.java`	Per-emplid rate limiting for OTP sends and session inits
`component/MfaEncryptionComponent.java`	AES+RSA encryption/decryption for request/response
`service/MfaOtpServiceImpl.java`	Service layer - calls Integration Broker
`framework/RequestValidator.java`	JWT bypass whitelist for MFA endpoints
`resources/uotm_constants.properties`	All MFA configuration properties

Scenarios

#	Scenario	Behavior
1	Happy path	init -> send -> verify (correct OTP) -> success
2	Wrong OTP, then correct	init -> send -> verify (wrong) -> verify (correct) -> success
3	3 wrong OTPs	init -> send -> verify x3 (all wrong) -> session invalidated
4	Resend OTP	init -> send -> verify (wrong) -> send (resend) -> verify (correct) -> success
5	Max resends	init -> send -> send -> send -> 4th send rejected ("max OTP requests reached")
6	Session expired	Any call after 5 min -> "session invalid/expired"
7	Session flood	6th `init_session` within 10 min -> rate limited
8	Correct OTP on 3rd attempt	Pre-check passes (counter=2 < 3), IB called, success returned
9	4th verify after 3 failures	Pre-check rejects immediately (counter=3 >= 3), IB not called